1. Подробное описание предметной области

1.1. Общая характеристика

Предметная область - деятельность электронного банка, связанная с обслуживанием клиентов и сотрудников в рамках единой информационной системы:

- регистрацией и идентификацией клиентов
- открытием и обслуживанием банковских счетов
- хранением информации о денежных средствах
- тестовым пополнением банковских счетов через внешнюю платёжную систему
- выполнением внутренних денежных переводов
- контролем лимитов
- разграничением доступа по ролям
- административным управлением пользователями, счетами и настройками системы
- выявлением мошеннических операций
- ручной проверкой подозрительных переводов
- формированием финансовой и антифрод-аналитики

1.2. Основные участники

Клиент банка

Клиент регистрируется, проходит авторизацию, управляет своими банковскими счетами, выполняет переводы и просматривает историю операций.

Антифрод-аналитик

Аналитик рассматривает операции, которые автоматическая система не смогла однозначно разрешить или отклонить. Он изучает причины срабатывания правил, историю клиента и граф связанных объектов, после чего принимает решение.

Администратор

Администратор управляет ролями и доступом пользователей, блокирует учётные записи и банковские счета, настраивает лимиты и антифрод-правила, а также просматривает аудит работы системы.

Внешние информационные системы

Для идентификации пользователей и управления ролями используется Keycloak. Также предполагается интеграция с сервисом определения географического положения по IP-адресу и с ЮKassa в тестовом режиме для имитации пополнения банковского счёта.

Ролевая модель

- `CLIENT` управляет собственными счетами, пополняет их, выполняет переводы и просматривает историю операций
- `ANALYST` работает с очередью ручных проверок, принимает решения и просматривает антифрод-аналитику
- `ADMIN` управляет пользователями, счетами, лимитами и правилами, а также просматривает административную аналитику и аудит

1.3. Основные сущности

В предметной области предусматриваются следующие сущности:

- профиль клиента
- роль пользователя в Keycloak
- устройство
- банковский счёт
- пополнение счёта
- перевод
- резерв денежных средств
- бухгалтерская проводка
- лимит
- антифрод-правило
- антифрод-проверка
- срабатывание правила
- антифрод-предупреждение
- решение аналитика
- запись аудита

1.4. Учёт денежных средств

Каждый банковский счёт принадлежит одному клиенту и имеет:

- уникальный номер
- состояние
- фактический баланс
- величину зарезервированных средств

- доступный баланс

```
Доступный баланс = Фактический баланс − Зарезервированные средства
```

Денежные средства не должны изменяться простым редактированием поля баланса. Каждая операция отражается бухгалтерскими проводками. Сумма проводок в рамках одной операции должна равняться 0.

Для перевода создаются как минимум две записи:

```
Счёт отправителя: −5000 рублей
Счёт получателя: +5000 рублей
```

Для пополнения создаётся отдельная заявка, связанная с идентификатором платежа во внешней системе. После подтверждения успешной тестовой оплаты баланс увеличивается не напрямую, а посредством бухгалтерских проводок между техническим счётом банка и счётом клиента. Повторная обработка одного и того же внешнего платежа не должна приводить к повторному зачислению.

1.5. Жизненный цикл перевода

Сначала средства резервируются. Это предотвращает создание нескольких одновременных переводов на сумму, превышающую доступный баланс. После резервирования операция асинхронно передаётся в антифрод-сервис.

Если перевод одобрен, резерв преобразуется в окончательные бухгалтерские проводки. Если операция отклонена, резерв освобождается. Для неоднозначной операции антифрод создаёт карточку ручной проверки. Пока аналитик не примет окончательное решение, перевод остаётся в денежном сервисе в состоянии `FRAUD_CHECK_PENDING`, а средства продолжают находиться в резерве.

Полный конечный автомат состояний, порядок межсервисного взаимодействия и восстановление после сбоев описаны в разделе 5.8 "Асинхронная обработка перевода".

1.6. Антифрод-проверка

Антифрод-модуль рассчитывает числовую оценку риска - `risk score`.

Примеры правил:

- перевод превышает обычную сумму операций клиента
- объём операций за период существенно отклоняется от обычного поведения клиента
- совершено много переводов за короткое время

- операция выполняется с нового устройства
- за короткое время изменилась геолокация клиента
- несколько клиентов используют одно устройство
- счёт получает средства от большого числа отправителей
- клиент переводит средства множеству новых получателей
- обнаружена циклическая цепочка переводов
- получатель ранее участвовал в подозрительных операциях

Каждое сработавшее правило вносит вклад в итоговую оценку, которая ограничивается диапазоном от 0 до 100:

```
riskScore = min(100, сумма весов сработавших правил)
```

Пример итогового решения:

```
0–29 -> APPROVED
30–69 -> REVIEW_REQUIRED
70–100 -> REJECTED
```

Вместе с оценкой сохраняются причины решения, чтобы результат был объяснимым.

Жёсткие дневные и месячные лимиты проверяются в `e-bank-money` до отправки операции в антифрод. Антифрод не дублирует эти ограничения, а оценивает отклонение операции от исторического поведения клиента.

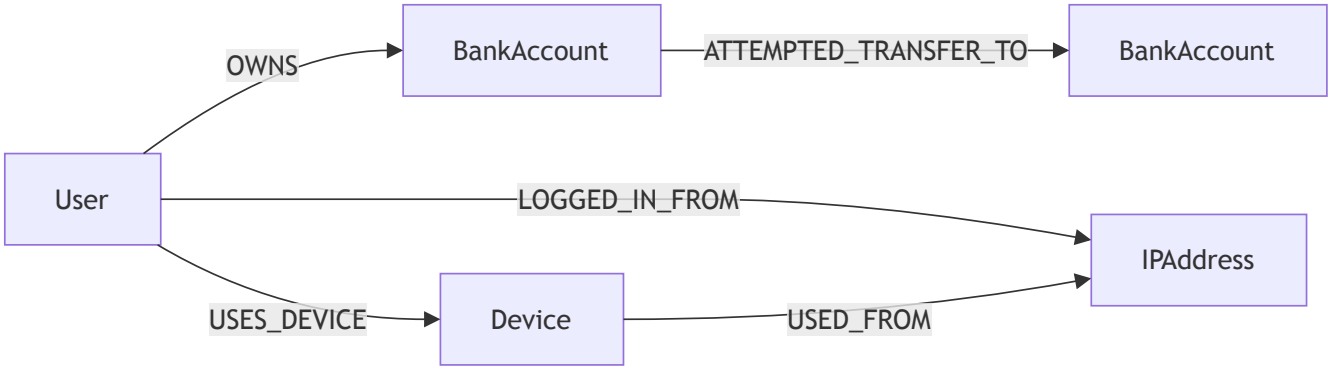
1.7. Графовый анализ

Антифрод представляет связи объектов в виде ориентированного неоднородного графа.

Вершинами являются:

- пользователь (`USER`)
- банковский счёт (`BANK_ACCOUNT`)
- устройство (`DEVICE`)
- IP-адрес (`IP_ADDRESS`)

Основные типы связей:



Связи содержат дополнительные свойства. Для использования устройства и IP-адреса сохраняются время первого и последнего появления, а также количество использований. Для попытки перевода сохраняются `transferId`, сумма, время, `riskScore`, решение антифрода, устройство и IP-адрес, с которых она была создана.

Перевод является отдельной сущностью денежного сервиса. При получении запроса на проверку антифрод добавляет в граф направленное ребро попытки перевода между банковскими счетами. Ребро сохраняется независимо от решения антифрода и дальнейшего результата проведения операции в `e-bank-money`. Поэтому граф отражает все наблюдавшиеся попытки взаимодействия между счетами, включая одобренные, отклонённые и переданные на ручную проверку операции. Значение `APPROVED` на ребре означает только решение антифрода и не гарантирует создание бухгалтерских проводок денежным сервисом.

Между двумя счетами может существовать несколько рёбер попыток перевода. При анализе правила могут учитывать все попытки либо фильтровать их по решению антифрода.

Графовая модель хранится в PostgreSQL антифрод-сервиса. При выполнении проверки необходимый подграф за ограниченный период загружается в память и представляется списком смежности. Для каждого антифрод-правила задаются допустимые типы вершин и рёбер, направление обхода, временной интервал и максимальная глубина.

Примеры алгоритмов:

- BFS для исследования ближайшего окружения счёта
- DFS для поиска цепочек переводов
- поиск циклов
- поиск компонент связности
- подсчёт входящей и исходящей степени вершины
- обнаружение схем `fan-in` и `fan-out`
- поиск кратчайшей связи между подозрительными объектами

1.8. Внешние интеграции

GeoIP-сервис

`e-bank-antifraud` обращается к внешнему GeoIP-сервису по HTTP, передаёт IP-адрес и получает страну, город и приблизительные географические координаты. Вызов выполняется с ограничением времени ожидания и небольшим числом повторных попыток, чтобы недоступность внешнего сервиса не блокировала антифрод-проверку на длительное время.

Полученные данные сохраняются в in-memory кеше антифрод-сервиса с IP-адресом в качестве ключа.

Если GeoIP-сервис временно недоступен, антифрод использует ранее сохранённое значение, в том числе допустимое устаревшее значение в пределах заданного срока. Если данных для IP-адреса в кеше нет, географические правила пропускаются, а остальные антифрод-правила продолжают выполняться. Отсутствие ответа GeoIP само по себе не является основанием для отклонения перевода.

Эти данные используются для выявления:

- входа из новой страны
- необычной географии операции
- физически невозможного перемещения

ЮKassa

Для имитации пополнения банковского счёта используется API ЮKassa в тестовом режиме. Денежный сервис создаёт заявку на пополнение со статусом `PENDING` и выполняет HTTP-запрос на создание тестового платежа с параметрами. Полученный от ЮKassa адрес `confirmation_url` передаётся клиентскому приложению для перехода на страницу подтверждения.

После возвращения пользователя в приложение клиент запрашивает состояние пополнения. Денежный сервис выполняет GET-запрос к API ЮKassa и получает актуальное состояние платежа. Пока платёж находится в состоянии `pending`, проверка может повторяться с увеличивающимся интервалом. При статусе `succeeded` сервис проверяет идентификатор и сумму платежа, идемпотентно создаёт бухгалтерские проводки и устанавливает заявке состояние `COMPLETED`. При статусе `canceled` заявка получает состояние `CANCELLED`, а баланс не изменяется.

2. Назначение информационной системы

Информационная система предназначена для комплексного обслуживания клиентов электронного банка, организации работы сотрудников с разными ролями, выполнения финансовых операций и контроля их безопасности.

Разрабатываемая система позволит решить следующие задачи:

1. Предоставлять единое клиентское приложение для работы со счетами и операциями
2. Централизованно выполнять идентификацию пользователей и управлять ролями
3. Разграничивать клиентские, аналитические и административные функции
4. Хранить данные клиентов и банковских счетов
5. Выполнять пополнения и внутренние денежные переводы
6. Обеспечить целостность финансового учёта
7. Предотвращать повторную и конкурентную обработку операций
8. Контролировать пользовательские лимиты
9. Предоставлять администраторам средства управления пользователями, счетами и настройками
10. Предоставлять аналитикам очередь ручной проверки подозрительных операций
11. Сохранять аудит действий клиентов и сотрудников
12. Формировать финансовую, административную и антифрод-аналитику
13. Обеспечить восстановление незавершённых операций после сбоя
14. Предоставлять технический мониторинг и централизованно собирать журналы работы системы
15. Интегрироваться с Keycloak, GeoIP-сервисом и тестовым API ЮKassa
16. Автоматически оценивать риск операций и сохранять причины решений
17. Выявлять подозрительные связи и цепочки переводов с помощью графового анализа

3. Требования к системе

3.1. Функциональные требования

Управление пользователями

Система должна:

1. Выполнять регистрацию и аутентификацию пользователей через Keycloak
2. Поддерживать роли клиента, аналитика и администратора
3. Позволять администратору блокировать и разблокировать пользователя
4. Завершать активные пользовательские сессии через Keycloak
5. Хранить сведения об известных устройствах
6. Фиксировать использование нового устройства или IP-адреса

Ролевой доступ

Система должна:

1. Предоставлять клиенту доступ только к собственным счетам, пополнениям и переводам
2. Предоставлять аналитику доступ к очереди ручных проверок, графу связей и антифрод-аналитике
3. Предоставлять администратору доступ к управлению пользователями, счетами, лимитами и правилами, а также к административной аналитике и аудиту
4. Запрещать выполнение операции при отсутствии необходимой роли

Управление счетами

Система должна:

1. Создавать банковские счета
2. Поддерживать несколько счетов одного пользователя
3. Поддерживать состояния активного, заблокированного и закрытого счёта
4. Запрещать операции по заблокированным счетам

Административное управление

Система должна:

1. Позволять администратору просматривать профили и состояния пользователей
2. Назначать и отзывать роли через Keycloak Admin API
3. Блокировать и разблокировать учётные записи через Keycloak Admin API
4. Блокировать и разблокировать банковские счета
5. Управлять лимитами и параметрами антифрод-правил
6. Фиксировать каждое административное изменение в аудите

Пополнение счетов

Система должна:

1. Создавать заявку на пополнение банковского счёта
2. Создавать тестовый платёж через API ЮKassa
3. Возвращать клиенту адрес страницы подтверждения платежа
4. Позволять клиенту запрашивать актуальное состояние пополнения
5. Получать актуальный статус платежа через API ЮKassa
6. Проверять идентификатор, сумму и валюту внешнего платежа

7. Идемпотентно зачислять средства при повторной проверке одного платежа
8. Отображать состояние и результат пополнения

Выполнение переводов

Система должна:

1. Создавать перевод с уникальным идентификатором
2. Проверять принадлежность счёта отправителю
3. Проверять состояние счетов
4. Проверять достаточность доступных средств
5. Проверять установленные лимиты
6. Резервировать средства перед антифрод-проверкой
7. Направлять операцию в антифрод на проверку
8. Выполнять перевод только после положительного решения
9. Создавать двойную бухгалтерскую запись
10. Освобождать резерв при отклонении
11. Защищаться от повторной обработки запроса
12. Позволять клиенту получать актуальное состояние перевода

Антифрод

Система должна:

1. Получать запросы на проверку
2. Применять активные антифрод-правила
3. Анализировать историю операций клиента
4. Анализировать операции во временном окне
5. Определять географию операции по IP
6. Поддерживать граф связанных объектов
7. Выявлять циклы и подозрительные группы
8. Рассчитывать `risk score`
9. Сохранять список сработавших правил
10. Формировать автоматический результат `APPROVED`, `REJECTED` или `REVIEW_REQUIRED`, но публиковать в Kafka только окончательное решение `APPROVED` или `REJECTED`
11. Обеспечивать ручную проверку
12. Позволять изменять веса и пороги правил
13. Хранить историю решений

Аналитика

Статистика пользователей и административных действий рассчитывается сервисом `e-bank-user`, финансовые показатели - сервисом `e-bank-money`, а статистика рисков и срабатываний правил - сервисом `e-bank-antifraud`. Клиентское приложение получает данные через Nginx из соответствующих сервисов и объединяет их для отображения в интерфейсе.

Система должна предоставлять:

- количество активных и заблокированных пользователей
- распределение банковских счетов по состояниям
- количество переводов за период
- денежный оборот
- долю завершённых и отклонённых переводов
- количество операций на ручной проверке
- распределение операций по `risk score`
- наиболее часто срабатывающие правила
- пользователей с наибольшим количеством предупреждений
- наиболее связанные устройства
- счета с аномальным входящим или исходящим потоком
- граф подозрительных связей
- среднее время антифрод-проверки

Аудит

Система должна фиксировать:

- входы в систему
- изменение ролей и блокировку пользователей
- создание и блокировку счетов
- создание переводов
- смену состояний перевода
- решения антифрода
- решения аналитиков
- изменения правил и лимитов

3.2. Нефункциональные требования

Целостность

- Создание перевода, резервирование средств, изменение состояния перевода и запись события в Outbox должны выполняться в одной транзакции PostgreSQL с

уровнем изоляции не ниже `READ COMMITTED` и блокировкой изменяемой записи счёта.
- Для каждой завершённой денежной операции сумма проводок должна быть равна нулю; нарушение должно приводить к откату транзакции.
- После фиксации любой транзакции должно выполняться условие `available_balance = balance − reserved_amount >= 0`.
- Ограничение `UNIQUE(user_id, idempotency_key)` должно обеспечивать создание не более одного перевода для одной пары пользователя и ключа идемпотентности.
- Идентификатор внешнего платежа ЮKassa должен быть уникальным: повторная проверка одного платежа не должна создавать дополнительные проводки.
- Операции `commit_transfer(...)` и `release_funds(...)` должны быть идемпотентными: повторный вызов не должен повторно изменять баланс, резерв или состояние перевода.

Надёжность

- Доступность HTTP API системы должна составлять не менее 99,5% за месяц.
- Обмен событиями должен выполняться по модели `at least once`; после успешной фиксации бизнес-транзакции событие должно сохраняться в Outbox и не теряться при недоступности Kafka или перезапуске сервиса.
- Все Kafka-потребители должны дедуплицировать сообщения по `eventId` через Transactional Inbox; повторная доставка не должна приводить к повторному бизнес-изменению.
- После восстановления Kafka накопленные события Outbox должны быть опубликованы не позднее 60 секунд при размере очереди до 10 000 событий.
- После перезапуска `e-bank-money` обработка переводов в состояниях `FRAUD_CHECK_PENDING`, `COMPLETING` и `RELEASING_FUNDS` должна возобновляться автоматически не позднее 60 секунд после готовности сервиса.

Производительность

- Получение списка счетов и первой страницы истории размером до 50 записей должно занимать не более 1 секунды для 95% запросов и не более 2 секунд для 99% запросов.
- Создание перевода, включая проверки, резервирование средств и возврат `transferId` со статусом `FRAUD_CHECK_PENDING`, должно занимать не более 3 секунд для 95% запросов и не более 6 секунд для 99% запросов.
- Автоматическая антифрод-проверка без ручного рассмотрения и без учёта времени недоступности внешнего GeoIP-сервиса должна завершаться не более чем за 5 секунд для 95% операций и не более чем за 10 секунд для 99% операций.

- Получение агрегированной аналитики за период до 31 дня должно занимать не более 4 секунд для 95% запросов.
- Система должна обслуживать не менее 50 одновременно активных пользователей и не менее 20 запросов в секунду в течение 30 минут; доля ответов HTTP `5xx` по внутренним причинам не должна превышать 1%.

Безопасность

- Весь внешний HTTP-трафик должен передаваться по HTTPS с TLS версии не ниже 1.2; прямой внешний доступ к backend-сервисам, Kafka и PostgreSQL должен быть запрещён сетевыми правилами.
- Регистрация, аутентификация, хранение учётных данных и управление сессиями должны выполняться только Keycloak; прикладные сервисы не должны хранить пароли пользователей.
- Каждый backend-сервис должен проверять подпись, `iss`, `aud`, `exp` и роли каждого JWT по открытым ключам Keycloak. Запрос с отсутствующим, просроченным или некорректным токеном должен отклоняться с HTTP `401`, а запрос без требуемой роли - с HTTP `403`.
- Проверка прав на объект должна выполняться на backend: пользователь с ролью `CLIENT` не должен получать доступ к счетам, переводам и пополнениям другого клиента даже при передаче их корректных идентификаторов.
- Срок действия access token должен составлять не более 15 минут. При блокировке пользователя Keycloak должен запретить выпуск и обновление токенов, а все активные сессии пользователя должны завершаться не позднее 60 секунд после команды администратора.
- Backend-сервисы должны принимать прокси-заголовки с исходным IP-адресом только от доверенного Nginx; входящие заголовки от внешнего клиента должны перезаписываться.
- Секреты Keycloak, PostgreSQL, Kafka и ЮKassa должны передаваться через переменные окружения и не должны присутствовать в исходном коде, образах контейнеров и журналах.
- В журналах запрещено сохранять пароли, access/refresh tokens, секретные ключи, полные номера счетов и полные платёжные реквизиты; номер счёта при журналировании должен маскироваться, кроме последних четырёх символов.
- Все входы, отказы в доступе, изменения ролей, блокировки, решения аналитика и изменения правил должны записываться в аудит с идентификатором субъекта, временем, типом действия, идентификатором объекта и результатом. Записи аудита должны быть недоступны для изменения через API и храниться не менее 365 дней.

Масштабируемость

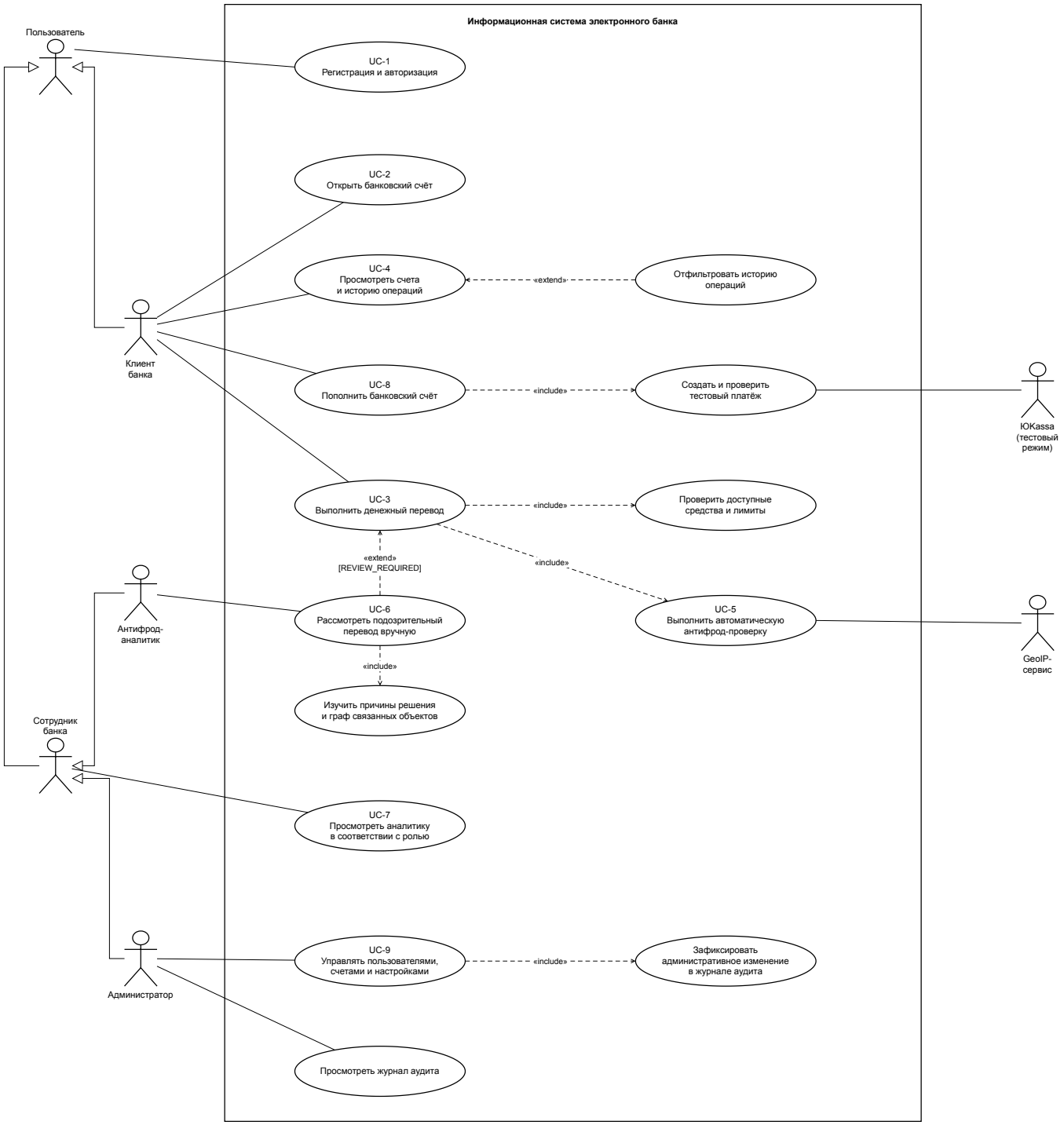
- `e-bank-user`, `e-bank-money` и `e-bank-antifraud` должны развёртываться и обновляться независимо; отказ одного сервиса не должен препятствовать запуску остальных сервисов.
- Прикладной сервис должен обращаться только к собственной схеме PostgreSQL. Доступ к данным другого сервиса разрешён только через версионированный HTTP API или Kafka-события.
- Каждый тип Kafka-события должен содержать `eventId`, `eventType`, `eventVersion`, `occurredAt` и идентификатор бизнес-объекта.
- Добавление необязательных полей в событие в пределах одной версии не должно нарушать работу существующих потребителей; удаление поля или изменение его семантики требует новой версии топика или контракта.

Наблюдаемость

- Каждый сервис должен предоставлять endpoint состояния с раздельными проверками готовности и работоспособности; проверка готовности должна отражать доступность обязательных зависимостей сервиса.
- Для каждого HTTP-запроса и Kafka-события должен передаваться `transferId`, позволяющий проследить обработку операции между сервисами.
- Сервисы должны публиковать следующие технические метрики: количество и длительность HTTP-запросов, долю ошибок, размер Outbox, задержку Kafka consumer, состояние пула соединений и число повторных попыток внешних вызовов.
- Должны публиковаться бизнес-метрики: число переводов по состояниям, сумма резервов, число решений антифрода по типам и размер очереди ручной проверки.

4. Основные прецеденты

4.1. Диаграмма прецедентов



4.2. Текстовые описания прецедентов

UC-1. Регистрация и авторизация

Основной участник: клиент.

Цель: получить доступ к системе.

Внешний участник: Keycloak.

Предусловия: пользователь не зарегистрирован либо имеет активную учётную запись.

Основной сценарий:

1. Клиентское приложение перенаправляет пользователя в Keycloak
2. Пользователь регистрируется либо вводит данные существующей учётной записи
3. Keycloak проверяет данные и создаёт пользовательскую сессию
4. Клиентское приложение получает код авторизации и обменивает его на токены
5. При первом обращении создаётся прикладной профиль, связанный с идентификатором `sub` из JWT
6. Фиксируются устройство, IP-адрес и время начала работы с приложением

Альтернативы:

- логин уже используется
- пароль не соответствует требованиям
- пользователь заблокирован
- указаны неверные данные
- вход выполнен с нового устройства

Результат: клиент авторизован либо получил обоснованный отказ.

UC-2. Создание банковского счёта

Основной участник: клиент.
Требуемая роль: `CLIENT`.
Предусловия: пользователь авторизован и не заблокирован.

Основной сценарий:

1. Система проверяет возможность открытия
2. Генерируется уникальный номер
3. Создаётся активный счёт с нулевым балансом
4. Действие записывается в аудит

Альтернативы:

- пользователь заблокирован
- достигнуто максимальное число счетов

Результат: создан новый банковский счёт.

UC-3. Выполнение денежного перевода

Основной участник: клиент.
Требуемая роль: `CLIENT`.

Внешний участник: GeoIP-сервис.

Предусловия:

- клиент авторизован
- счёт принадлежит клиенту
- счета активны
- доступных средств достаточно

Основной сценарий:

1. Клиент выбирает счёт списания
2. Указывает получателя и сумму
3. Nginx направляет запрос в `e-bank-money`, передавая IP-адрес в доверенном прокси-заголовке
4. Денежный сервис проверяет JWT Keycloak, извлекает идентификатор пользователя и роли, а также формирует контекст устройства и IP-адреса
5. Денежный сервис проверяет принадлежность и состояние счетов, доступный баланс и лимиты
6. Денежный сервис создаёт перевод и резервирует средства
7. Запрос на проверку асинхронно передаётся в антифрод
8. Клиент получает `transferId` и текущее состояние `FRAUD_CHECK_PENDING`
9. Антифрод рассчитывает риск и формирует автоматический результат
10. При `REVIEW_REQUIRED` антифрод создаёт карточку ручной проверки, а денежный сервис продолжает ожидать окончательное решение
11. После получения окончательного решения денежный сервис проводит перевод либо освобождает резерв
12. Клиент запрашивает состояние перевода по `transferId` и получает актуальный результат

Альтернативы:

- недостаточно средств - перевод не создаётся
- превышен лимит - операция отклоняется
- требуется ручная проверка - антифрод создаёт карточку для аналитика, а перевод остаётся в `FRAUD_CHECK_PENDING`
- антифрод отклоняет перевод - резерв освобождается
- запрос повторен - возвращается ранее созданный перевод

Результат: запрос на перевод принят, а его актуальное состояние доступно клиенту по `transferId`; перевод завершается, отклоняется либо ожидает решения.

UC-4. Просмотр счетов и истории

Основной участник: клиент.
Требуемая роль: `CLIENT`.

Основной сценарий:

1. Клиент открывает список счетов
2. Система возвращает баланс и зарезервированные средства
3. Клиент выбирает счёт
4. Получает историю переводов и проводок
5. Применяет фильтры по периоду, сумме, типу и статусу

Результат: клиент получает информацию только по принадлежащим ему счетам.

UC-5. Автоматическая антифрод-проверка

Связь с другими прецедентами: включаемый прецедент UC-3 "Выполнение денежного перевода".

Внешний участник: GeoIP-сервис.

Предусловия: получен запрос на антифрод-проверку.

Основной сценарий:

1. Загружается история операций
2. Определяется география IP-адреса
3. Применяются правила временного анализа
4. Выполняется графовый анализ
5. Рассчитывается итоговый `risk score`
6. Сохраняются результат и сработавшие правила
7. При `APPROVED` или `REJECTED` публикуется окончательное решение

Альтернативы:

- при `REVIEW_REQUIRED` создаётся карточка ручной проверки, а решение в Kafka не публикуется до действия аналитика
- уже обработанный запрос пропускается
- GeoIP-сервис недоступен - применяется сохранённое значение или правило работает без географии
- произошла временная ошибка - запрос обрабатывается повторно

UC-6. Ручная проверка операции

Основной участник: антифрод-аналитик.
Требуемая роль: `ANALYST`.
Предусловия: перевод отправлен на ручную проверку.

Основной сценарий:

1. Аналитик открывает очередь операций
2. Выбирает перевод
3. Просматривает сумму, клиента и историю
4. Изучает сработавшие правила
5. Просматривает граф связей
6. Подтверждает или отклоняет операцию
7. Решение сохраняется
8. Перевод завершается или освобождается резерв.

Результат: операция получает окончательное решение, действия аналитика записаны в аудит.

UC-7. Просмотр аналитики

Основные участники: аналитик и администратор.
Требуемая роль: `ANALYST` или `ADMIN`.

Основной сценарий:

1. Пользователь выбирает период
2. Система проверяет роль и формирует доступные для неё агрегированные показатели
3. Отображаются количество и оборот переводов
4. Отображается статистика антифрод-решений
5. Строятся графики по времени и правилам
6. Пользователь может перейти от агрегата к доступным ему операциям
7. Отдельно отображается граф подозрительных связей

Результат: пользователь получает аналитические данные в соответствии со своей ролью.

UC-8. Пополнение банковского счёта

Основной участник: клиент.
Требуемая роль: `CLIENT`.

Внешний участник: ЮKassa.

Предусловия: клиент авторизован, банковский счёт активен.

Основной сценарий:

1. Клиент выбирает банковский счёт и указывает сумму пополнения
2. Система создаёт заявку на пополнение со статусом `PENDING`
3. Денежный сервис создаёт тестовый платёж через API ЮKassa
4. Клиент переходит на страницу подтверждения и выполняет тестовую оплату
5. После возвращения в приложение клиент запрашивает состояние пополнения
6. Денежный сервис получает актуальный статус платежа через API ЮKassa
7. При статусе `succeeded` система проверяет идентификатор, сумму и валюту, создаёт проводки и устанавливает `COMPLETED`

Альтернативы:

- платёж имеет статус `pending` - заявка остаётся в `PENDING`
- платёж имеет статус `canceled` - заявка получает `CANCELLED`
- ЮKassa недоступна - состояние заявки не изменяется, проверка повторяется позднее
- состояние запрошено повторно - средства повторно не зачисляются
- идентификатор или сумма не совпадают - баланс не изменяется

Результат: после подтверждённой тестовой оплаты средства однократно зачислены на банковский счёт.

UC-9. Административное управление

Основной участник: администратор.
Требуемая роль: `ADMIN`.
Предусловия: администратор авторизован.

Основной сценарий:

1. Администратор открывает панель управления
2. Просматривает пользователей, их роли, счета, лимиты или антифрод-правила
3. Выполняет разрешённое изменение
4. Система проверяет роль администратора
5. При управлении доступом `e-bank-user` выполняет необходимую операцию через Keycloak Admin API
6. Изменение сохраняется, а действие записывается в аудит

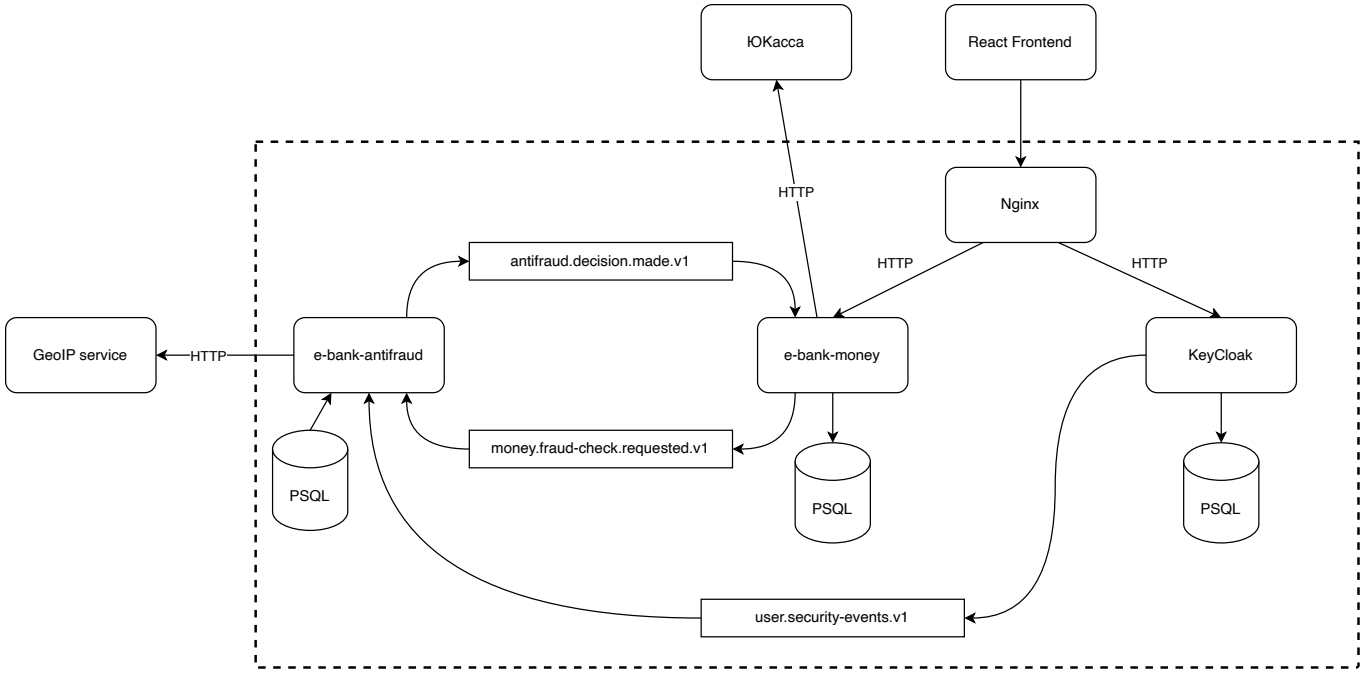
Альтернативы:

- операция запрещена текущей ролью
- пользователь или объект не найден
- Keycloak временно недоступен - состояние доступа не изменяется
- изменение нарушает ограничение целостности - операция отклоняется

Результат: административное изменение применено и зафиксировано в журнале аудита.

5. Архитектура системы

5.1. Общая схема



5.2. Nginx

Назначение:

- единая точка входа
- раздача собранного React-приложения
- маршрутизация HTTP-запросов к backend-сервисам и Keycloak
- завершение TLS-соединений
- передача исходного IP-адреса через доверенные прокси-заголовки
- базовые ограничения размера и частоты запросов

Пример маршрутизации:

```
/api/users/** -> e-bank-user
/api/money/** -> e-bank-money
```

```
/api/antifraud/** -> e-bank-antifraud
/auth/** -> Keycloak
```

5.3. Keycloak

Назначение:

- регистрация и аутентификация пользователей
- хранение учётных данных
- управление пользовательскими сессиями
- выпуск и обновление JWT
- хранение ролей `CLIENT`, `ANALYST` и `ADMIN`
- блокировка учётных записей
- использование собственной схемы PostgreSQL

5.4. `e-bank-user`

Назначение:

- прикладные профили клиентов, связанные с `sub` из Keycloak
- устройства
- аудит начала работы с приложением и использования устройств
- административное управление доступом через Keycloak Admin API
- публикация событий безопасности
- статистика пользователей и административных действий
- Outbox

Сервис не хранит пароли и не выпускает JWT. При первом обращении авторизованного пользователя он создаёт профиль, связанный с идентификатором `sub` из токена. События регистрации профиля, добавления устройства и использования нового IP-адреса надёжно публикуются через Transactional Outbox.

5.5. `e-bank-money`

Назначение:

- банковские счета
- остатки и резервы
- переводы
- проводки
- лимиты

- заявки на пополнение и интеграция с ЮKassa
- конечный автомат перевода
- проведение перевода и освобождение резерва
- Outbox и Inbox

Только этот сервис имеет право изменять денежные данные.

5.6. `e-bank-antifraud`

Назначение:

- антифрод-правила
- временной анализ операций
- расчёт `risk score`
- история проверок
- граф связанных объектов
- автоматические решения
- очередь ручной проверки
- аналитика рисков, правил и графовых связей
- Outbox и Inbox

Антифрод не изменяет баланс, он только выдает решение.

5.7. Взаимодействие компонентов

Аутентификация и маршрутизация

React-приложение направляет пользователя в Keycloak для регистрации или входа. После получения JWT клиент отправляет запросы через Nginx. Nginx выбирает целевой backend-сервис по пути запроса и передаёт ему запрос, но не принимает решений о правах пользователя. Каждый сервис самостоятельно проверяет JWT и доступность операции для роли пользователя.

Создание перевода

Клиент передаёт идентификаторы счетов, сумму и ключ идемпотентности. Nginx направляет запрос в `e-bank-money` и передаёт исходный IP-адрес через доверенный прокси-заголовок. Денежный сервис проверяет JWT Keycloak, извлекает идентификатор пользователя и его роли, после чего формирует контекст пользователя, устройства и IP-адреса.

Денежный сервис самостоятельно получает владельцев и состояния счетов, баланс, активные резервы и лимиты, после чего создаёт `transferId`.

Блокировка и события безопасности

Администратор выполняет блокировку через интерфейс системы. `e-bank-user` обращается к Keycloak Admin API, отключает учётную запись и завершает активные пользовательские сессии. После блокировки Keycloak запрещает новый вход, выпуск и обновление токенов.

`e-bank-user` публикует события создания профиля, добавления устройства и начала работы с приложением с нового IP-адреса. Антифрод обрабатывает их и обновляет локальный граф связей пользователей, устройств и IP-адресов.

Пополнение через ЮKassa

`e-bank-money` создаёт тестовый платёж через REST API ЮKassa и передаёт клиенту адрес страницы подтверждения. После возвращения пользователя клиент периодически запрашивает состояние заявки через Nginx, а денежный сервис получает актуальный статус платежа из ЮKassa.

Зачисление выполняется только при статусе `succeeded` после проверки идентификатора, суммы и валюты. Повторное зачисление предотвращается уникальностью идентификатора внешнего платежа и транзакционной обработкой операции.

Проверка перевода

После резервирования средств `e-bank-money` отправляет в Kafka запрос на антифрод-проверку. Он содержит `transferId`, идентификаторы пользователя и счетов, сумму, устройство и IP-адрес.

Антифрод анализирует правила и граф связей отправителя и получателя и рассчитывает `risk score`. При `APPROVED` или `REJECTED` он публикует окончательное решение. При `REVIEW_REQUIRED` создаётся карточка ручной проверки, а решение публикуется только после действия аналитика. После получения окончательного решения денежный сервис либо проводит перевод, либо освобождает резерв.

Используются топики:

- `user.security-events.v1`;
- `money.fraud-check.requested.v1`;
- `antifraud.decision.made.v1`.

События одного перевода связываются по `transferId`, а повторная доставка обнаруживается по уникальному `eventId`.

Аналитика

`e-bank-user` рассчитывает статистику пользователей и административных действий. `e-bank-money` рассчитывает количество переводов, денежный оборот и распределение операций по состояниям. `e-bank-antifraud` формирует статистику решений, значений `risk score`, срабатываний правил и графовых связей.

React-приложение через Nginx запрашивает необходимые агрегаты из трёх сервисов и объединяет результаты при отображении.

5.8. Асинхронная обработка перевода

`e-bank-money` является владельцем перевода и всех денежных данных. Антифрод-проверка выполняется асинхронно сервисом `e-bank-antifraud`, а надёжность взаимодействия обеспечивается конечным автоматом состояний, Transactional Outbox, Transactional Inbox и идемпотентной обработкой. Распределённая транзакция для этого процесса не требуется.

Первый этап: создание перевода и резервирование

После получения запроса `e-bank-money` выполняет одну локальную транзакцию:

1. Проверяет ключ идемпотентности.
2. Проверяет принадлежность и состояние счетов.
3. Блокирует запись счёта отправителя.
4. Проверяет доступный баланс и установленные лимиты.
5. Создаёт запись перевода и резервирует средства с помощью функции `reserve_funds(...)`.
6. Устанавливает состояние перевода `FRAUD_CHECK_PENDING`.
7. Добавляет событие `FraudCheckRequested` в таблицу Outbox.
8. Фиксирует транзакцию.

После резервирования доступный баланс уменьшается, но окончательные бухгалтерские проводки ещё не создаются. Запись перевода и событие Outbox содержат общий `transferId`. После фиксации транзакции Outbox Publisher отправляет событие в топик `money.fraud-check.requested.v1`. При ошибке публикация повторяется.

Второй этап: антифрод-проверка

`e-bank-antifraud` получает `FraudCheckRequested` из Kafka. Сначала сервис получает географические данные из in-memory кеша или, при их отсутствии, обращается к GeoIP API. Внешний HTTP-запрос выполняется до открытия транзакции PostgreSQL.

После этого `e-bank-antifraud` в одной локальной транзакции:

1. Регистрирует `eventId` в Inbox
2. Создаёт запись антифрод-проверки
3. Добавляет сведения о попытке перевода в графовую модель
4. Применяет временные и графовые правила
5. Рассчитывает `risk score`
6. Сохраняет решение и причины его принятия
7. Если получено окончательное решение APPROVED или REJECTED, добавляет FraudDecision в Outbox
8. Если требуется ручная проверка, создаёт карточку проверки и не публикует решение до действия аналитика

Окончательное решение публикуется в топик `antifraud.decision.made.v1`. Сообщение содержит `eventId`, `transferId`, решение, источник решения `AUTOMATIC` или `ANALYST`, `riskScore`, сработавшие правила и время принятия решения.

Если автоматическая проверка требует участия аналитика, антифрод не публикует промежуточное решение в топик `antifraud.decision.made.v1`. Он создаёт карточку ручной проверки, сохраняет причины срабатывания правил и ожидает решения аналитика. В это время перевод остаётся в `e-bank-money` в состоянии `FRAUD_CHECK_PENDING`, а средства продолжают находиться в резерве.

После рассмотрения аналитик выбирает `APPROVED` или `REJECTED`. Антифрод сохраняет решение и запись аудита, после чего через Outbox публикует итоговое событие `FraudDecision` с источником `ANALYST`. Денежный сервис получает решение и завершает перевод либо освобождает резерв.

Третий этап: обработка решения

`e-bank-money` получает событие `FraudDecision`, регистрирует его в Inbox, блокирует запись перевода и проверяет допустимость перехода состояния. Запись Inbox и изменение состояния перевода выполняются в одной локальной транзакции.

Антифрод публикует только окончательные решения:

- `APPROVED` переводит операцию из `FRAUD_CHECK_PENDING` в `COMPLETING`;
- `REJECTED` переводит операцию из `FRAUD_CHECK_PENDING` в `RELEASING_FUNDS`.

Решение может иметь источник `AUTOMATIC` или `ANALYST`. Источник сохраняется для аудита, но не влияет на переход состояния. Если требуется ручная проверка, антифрод не публикует промежуточное событие: перевод остаётся в `FRAUD_CHECK_PENDING` до принятия окончательного решения аналитиком.

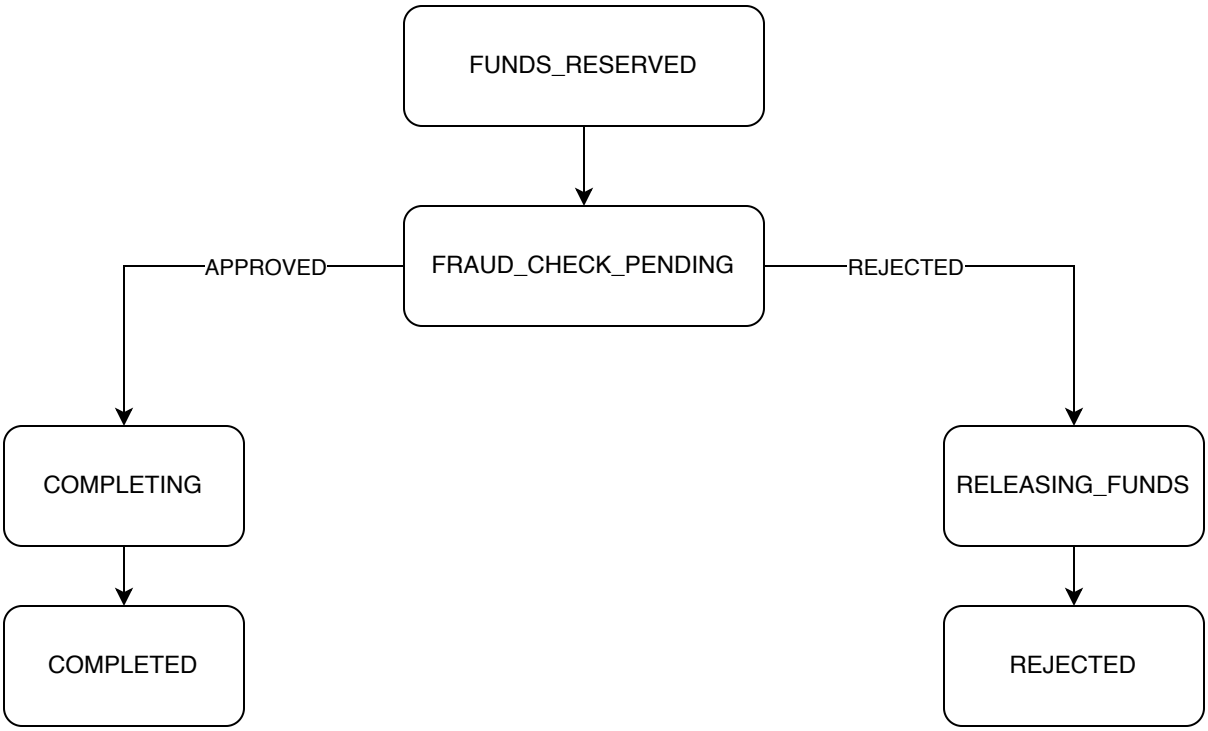
События дедуплицируются по `eventId`.

Четвёртый этап: проведение или освобождение резерва

Для операции в `COMPLETING` денежный сервис в одной локальной транзакции вызывает `commit_transfer(...)`, погашает резерв, создаёт дебетовую и кредитовую проводки и устанавливает состояние `COMPLETED`.

Для операции в `RELEASING_FUNDS` сервис в одной локальной транзакции вызывает `release_funds(...)`, освобождает резерв и устанавливает состояние `REJECTED`. Причина отклонения сохраняется вместе с переводом.

Конечный автомат состояний



Обработка сбоев и восстановление

Состояние обработки хранится в таблице переводов. После перезапуска `e-bank-money` находит незавершённые операции и продолжает их обработку:

- `FRAUD_CHECK_PENDING` — ожидает окончательное решение антифрода; если запрос ещё не был опубликован, Outbox Publisher повторяет его отправку;
- `COMPLETING` — повторно вызывает идемпотентную функцию `commit_transfer(...)`;
- `RELEASING_FUNDS` — повторно вызывает идемпотентную функцию `release_funds(...)`.

Если антифрод временно недоступен, опубликованный запрос сохраняется в Kafka и обрабатывается после восстановления потребителя. Если Kafka недоступна, событие остаётся в Outbox и отправляется повторно. Возможный дубликат запроса или решения

обнаруживается по `eventId` в Inbox и не приводит к повторному изменению состояния или баланса.

5.9. Outbox, Inbox и идемпотентность

Обработка сообщений Kafka настраивается по модели `at least once`: сообщение сохраняется до подтверждения обработки и при сбое может быть доставлено повторно. Для надёжной работы используются Transactional Outbox, Transactional Inbox и идемпотентные операции.

Transactional Outbox

Нельзя перевести операцию в состояние ожидания антифрода и независимо отправить соответствующее сообщение:

```
Установить FRAUD_CHECK_PENDING в PostgreSQL
    < возможен сбой >
Отправить FraudCheckRequested в Kafka
```

Создание перевода, резервирование средств, установка состояния `FRAUD_CHECK_PENDING` и добавление `FraudCheckRequested` в Outbox выполняются в одной транзакции PostgreSQL. Если транзакция откатывается, не сохраняются ни резерв, ни перевод, ни исходящее событие. Если транзакция зафиксирована, запрос гарантированно остаётся в Outbox и будет опубликован после восстановления Kafka.

Отдельный Outbox Publisher периодически выбирает записи со статусом `NEW`, отправляет их в Kafka и устанавливает `PUBLISHED`. Если Kafka временно недоступна, событие остаётся в таблице и отправляется позднее.

Transactional Inbox

Consumer в одной локальной транзакции сохраняет `eventId` в Inbox и выполняет соответствующее бизнес-изменение. На `eventId` устанавливается ограничение уникальности. Если сообщение уже зарегистрировано, повторная обработка пропускается.

Идемпотентность HTTP-запросов

При создании перевода клиент передаёт ключ идемпотентности.

В базе устанавливается ограничение:

```
UNIQUE(user_id, idempotency_key)
```

Если клиент повторит запрос из-за сетевой ошибки, новый перевод не создаётся. Система возвращает уже существующую операцию.

6. Технологии

Для реализации системы предлагается следующий стек:

- **Frontend:** React, TypeScript
- **Backend:** Java, Spring Boot, Spring MVC, Spring Security, Spring Kafka
- **База данных:** PostgreSQL
- **Идентификация и роли:** Keycloak, OAuth 2.0, OpenID Connect
- **Внешний доступ и маршрутизация:** Nginx
- **Взаимодействие сервисов:** REST API и Kafka
- **Внешние интеграции:** GeoIP API и REST API ЮKassa в тестовом режиме
- **Надёжность:** Transactional Outbox/Inbox, идемпотентная обработка, retry
- **Документирование API:** Swagger
- **Тестирование:** JUnit, Spring Boot Test
- **Развёртывание:** Docker Compose